Installation locale d'une pharmacie d'hôpital — mode hors-ligne et synchronisation à double sens

Mpangi Pharma — dossier technique. Date : 14 octobre 2026.

1. En bref

Le besoin. La machine de la pharmacie de l'hôpital doit fonctionner même sans internet, avec toutes ses données en local. Dès que la connexion revient, elle se synchronise avec le serveur en ligne, dans les deux sens. Pendant ce temps, les téléphones connectés à internet utilisent directement le serveur en ligne.

Ce qui est possible. Oui, c'est réalisable : un « nœud local » (l'application + une base de données sur la machine) qui se réplique avec le cloud.

Ce qui n'existe pas encore, et que je ne veux pas vous promettre à tort. La synchronisation à double sens **n'est pas construite**. Aujourd'hui l'application ne sait que : - vendre hors-ligne (les ventes sont gardées dans le navigateur puis envoyées au serveur) ; - aucune copie de la base ne tourne en local.

Ce document donne : l'état réel de l'application (vérifié dans le code et la base), l'architecture à installer, la procédure d'installation, la conception détaillée de la synchronisation (y compris les risques), un plan par phases avec estimation, et les décisions dont j'ai besoin pour commencer.

Mon conseil. Ne pas installer en production un nœud local tant que la synchronisation n'a pas été construite **et testée avec de vraies coupures** sur une copie de la base. Une mauvaise fusion de stock coûte plus cher qu'une coupure de quelques heures. Voir le plan au chapitre 8 : un pilote sans synchronisation d'abord, puis la synchronisation par étapes.

2. Ce qui existe aujourd'hui (vérifié)

Sujet	État réel
Ventes hors-ligne	Oui. Produits copiés dans le navigateur (IndexedDB), ventes enregistrées localement avec un ticket provisoire, puis envoyées avec le bouton **Synchroniser ventes**. Le serveur recalcule alors le stock (lots les plus proches de l'expiration d'abord) et attribue le vrai numéro de facture.
Autres opérations hors-ligne	**Non** : entrées de stock, corrections, création de produits, inventaire, finances, utilisateurs demandent internet.
Démarrage à froid sans internet	**Non.** Le service worker ne met pas l'application en cache. Si le PC redémarre ou si on ferme l'onglet pendant une coupure, l'application ne s'ouvre plus (message « Hors ligne »). Le mode hors-ligne actuel ne marche que si la page était déjà ouverte avant la coupure.
Déconnexion	Se déconnecter efface les données hors-ligne de l'appareil (ventes non envoyées comprises).
Base de données locale	Aucune. Tout repose sur Supabase en ligne.

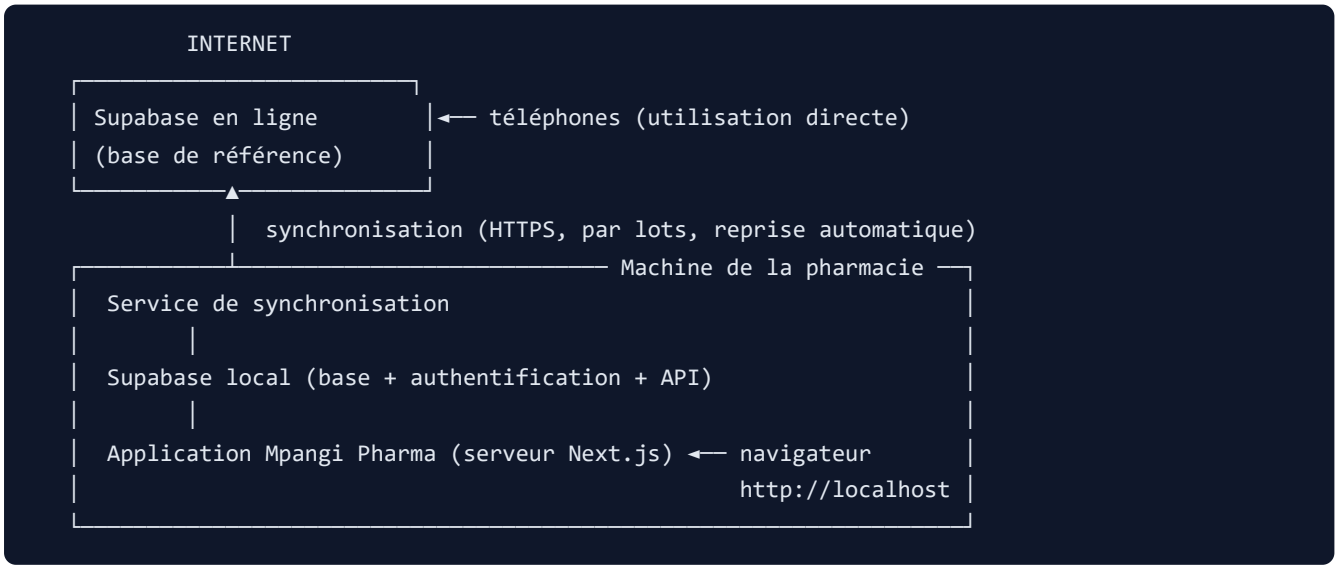
Taille actuelle de votre base (3 pharmacies) : 38 produits, 38 lots, 77 mouvements de stock, 20 ventes, 22 profils. Le catalogue national contient 6 237 produits. La base est petite : une copie locale et une synchronisation seront rapides.

3. Deux architectures possibles

	A. Poste renforcé (sans base locale)	B. Nœud local complet (ce que vous décrivez)
Principe	L'application est mise en cache dans le navigateur ; les opérations faites hors-ligne sont rangées dans une file et rejouées au retour d'internet.	L'application **et** une base Supabase tournent sur la machine. Une synchronisation copie les changements dans les deux sens.
Démarrage sans internet	Oui, une fois l'application installée (à construire).	Oui, naturellement.
Fonctions hors-ligne	Ventes + quelques opérations de stock (à construire une par une). Rapports, inventaire, finances : seulement sur ce qui est en cache.	Toutes, comme en ligne.
Risque de conflit de données	Faible : le serveur reste seul juge, il rejoue les opérations.	**Réel** : deux bases modifient le même stock. Demande une conception soignée (chapitre 7).
Maintenance sur place	Aucune (un navigateur).	Oui : Docker, sauvegardes, supervision, mises à jour.
Effort estimé	2 à 3 semaines	5 à 7 semaines (indicatif)

Recommandation. Si les coupures durent quelques heures et que vendre + recevoir du stock suffit, **A** est plus sûr et moins cher. Si l'hôpital doit travailler plusieurs jours sans internet avec toutes les fonctions, ou si la direction exige « tout en local », il faut **B**. Votre demande décrit B ; ce dossier détaille B, et signale ce que A apporterait.

4. Architecture cible (option B)



Éléments : la base Supabase locale (même schéma que le cloud), l'application Next.js locale, le service de synchronisation, des sauvegardes automatiques et un onduleur.

5. Matériel et logiciels

Machine (dédiée, pas un PC de bureautique partagé) : - 16 Go de RAM recommandés (8 Go minimum), SSD 256 Go ou plus, processeur 4 cœurs ; - Windows 11 Pro (avec WSL2 et Docker Desktop) **ou** Ubuntu Server 22.04/24.04 (plus stable pour un service continu) ; - **onduleur** (la coupure électrique est le risque n°1 pour une base de données) ; - disque externe ou seconde machine pour les sauvegardes ; - chiffrement du disque (BitLocker) : ce sont des données de santé.

Logiciels : Docker (Desktop ou Engine), Supabase CLI, Node.js 20.9 ou plus récent (version LTS), Git, un navigateur à jour (Chrome ou Edge).

La pile Supabase locale consomme environ 2 à 3 Go de mémoire.

6. Procédure d'installation (option B)

> Ces commandes sont les commandes officielles des outils. Je ne les ai **pas encore exécutées de bout en bout** sur une machine de test : une répétition complète sur un PC neuf est la première étape du pilote (chapitre 8).

Étape 0 — Préparer la machine

1. Installer Windows/Ubuntu à jour, activer le chiffrement du disque, régler l'**heure automatique (NTP)** : la synchronisation dépend d'horloges justes.
2. Installer Docker, Node.js LTS, Git et le Supabase CLI.
3. Brancher l'onduleur et tester une coupure électrique contrôlée.

Étape 1 — Récupérer l'application

```
git clone https://github.com/aksantictech/mpangi-pharma.git
cd mpangi-pharma
npm ci
```

Étape 2 — Démarrer Supabase en local

```
supabase init        # une seule fois
supabase start       # lance base, authentification, API (ports 54321 API, 54322 base, 54323
Studio)
```

`supabase start` affiche l'URL de l'API locale et ses clés locales (`anon` et `service_role`). Ces clés sont **différentes** de celles du cloud.

Étape 3 — Copier la base en ligne, « à jour »

Le schéma n'est que partiellement versionné dans `supabase/migrations` (0000 à 0005) : la base en ligne fait foi. Il faut donc l'exporter.

1. **Schéma** (fonctions, politiques de sécurité, vues, déclencheurs). Il faut le mot de passe de la base (Supabase > Project Settings > Database). Ne l'envoyez à personne par message :

```
supabase db dump --db-url "postgresql://postgres:<MOT_DE_PASSE>@<HOTE_CLOUD>:5432/postgres" -f
schema.sql
```

1. **Données de la pharmacie de l'hôpital seulement.** La base en ligne contient les données de plusieurs pharmacies : on ne doit copier sur la machine locale que les lignes de **cette** pharmacie (filtre `pharmacy_id`), plus le catalogue national et les comptes utilisateurs concernés. Un `pg_dump` brut copierait tout : à éviter. L'export filtré est un script à écrire (chapitre 8, phase 2).
2. **Comptes** : les utilisateurs vivent dans le schéma d'authentification (`auth.users`). Les copier avec leurs mots de passe chiffrés permet de se connecter en local avec les mêmes identifiants.
3. **Restauration locale** :

```
psql "postgresql://postgres:postgres@127.0.0.1:54322/postgres" -f schema.sql
# puis import des données filtrées
```

Étape 4 — Configurer l'application locale

Créer `.env.local` avec l'URL et les clés **locales** (étape 2) :

```
NEXT_PUBLIC_SUPABASE_URL=http://127.0.0.1:54321
NEXT_PUBLIC_SUPABASE_ANON_KEY=<clé anon locale>
SUPABASE_SERVICE_ROLE_KEY=<clé service_role locale>
```

Ne jamais mettre la clé `service_role` du **cloud** sur la machine de la pharmacie.

Étape 5 — Construire et lancer en service

```
npm run build
npm start           # écoute sur le port 3000
```

Le faire redémarrer tout seul au démarrage de la machine (service Windows, `pm2`, ou conteneur Docker avec `restart: always`).

Étape 6 — Poste de travail

Raccourci du navigateur vers `http://localhost:3000`, plein écran, imprimante de tickets configurée. Les autres PC de l'hôpital sur le même réseau local peuvent utiliser `http://<adresse-ip-de-la-machine>:3000` (réseau interne uniquement, jamais exposé à internet).

Étape 7 — Sauvegardes

Sauvegarde automatique quotidienne de la base locale vers un disque externe, avec un test de restauration chaque mois. Une sauvegarde jamais restaurée n'est pas une sauvegarde.

Étape 8 — Recette (voir chapitre 10)

7. Conception de la synchronisation à double sens

7.1 Principes

- Le **cloud reste la référence**. Chaque machine locale a un identifiant de nœud.
- On synchronise des **événements** (« une vente a eu lieu », « 20 boîtes reçues »), pas des états. Copier le stock « quantité = 7 » d'un côté vers l'autre écraserait les ventes de l'autre côté.
- Chaque changement est identifié par un UUID : l'envoyer deux fois ne le crée qu'une fois (idempotence), ce qui permet de reprendre après une coupure en plein envoi.

7.2 Classement des tables (d'après le schéma réel)

Famille	Tables	Règle
Événements en ajout seul	`sales`, `sale_items`, `stock_movements`, `kit_requisitions`, `kit_requisition_items`, `expenses`, `audit_logs`, `auth_events`, `subscription_payments`	Copiés tels quels dans les deux sens, une seule fois (clé = UUID). Pas de conflit sur la création. Quelques champs d'état changent ensuite (annulation d'un prélèvement, mode de paiement d'une vente) : sur ces champs seulement, le plus récent gagne.
État dérivé critique	`product_batches` (colonne `quantity_available`)	**Jamais copié ligne à ligne.** Chaque côté applique les mouvements reçus comme des variations (+/-) une seule fois par événement.
Fiches modifiables	`products`, `product_categories`, `suppliers`, `customers`, `pricing_rules`, `pharmacy_payment_methods`, `pharmacy_settings`, `pharmacy_members`, `profiles`, `pharmacy_opening_hours`, `pharmacies`	Dernière modification gagnante (`updated_at`), avec suppression « douce » (archivage) plutôt que suppression réelle.
Cloud vers local seulement	`national_products` (6 237 lignes), `pharmacy_subscriptions`, `platform_admins`, `platform_payment_settings`	Descendent du cloud, jamais l'inverse.

7.3 Points de blocage à traiter **avant** de synchroniser

1. **Chaque changement de quantité doit laisser une trace.** Vérification faite sur votre base : sur 34 produits, 2 ont un stock qui ne correspond pas à la somme des mouvements (Doliprane chez ELIKYA : stock 0 mais mouvements -2 ; Ibucap Forte chez DOREAH : stock 10 mais aucun mouvement). Ce sont des changements faits sans mouvement (stock initial, retrait d'un lot). Tant que ça existe, la synchronisation par événements dérive. Il faut ajouter les mouvements manquants (mouvement d'ouverture, mouvement de retrait de lot) côté base.
2. **Numéros de facture.** Chaque côté génère `FAC-…0001`. Deux ventes différentes auraient le même numéro. Solution : un préfixe par nœud (ex. `H1-` pour l'hôpital) via le paramètre de préfixe de

facture. À confirmer : la présence d'une contrainte d'unicité sur `sales.invoice_number`.

3. **Suppressions.** Une ligne supprimée d'un côté doit disparaître de l'autre : il faut une marque de suppression (`deleted_at`) ou une table des suppressions.
4. **Boucles.** Quand la synchronisation écrit une ligne reçue, les déclencheurs de la base ne doivent pas la renvoyer. Techniquement : désactiver les déclencheurs de réplication pendant l'application des changements distants.
5. **Comptes utilisateurs et clés.** Les jetons de connexion locaux et cloud ne sont pas interchangeables : on se connecte séparément sur chaque serveur, avec les mêmes identifiants.

7.4 Le vrai risque métier : deux ventes du dernier article

Si la machine locale (coupée d'internet) et un téléphone (en ligne) vendent chacun la dernière boîte du même produit, le stock devient négatif après fusion. Aucun système ne peut l'éviter sans connexion. Règles proposées : - la fusion **accepte** les deux ventes (elles ont réellement eu lieu) et **signale** l'écart dans l'écran de synchronisation et sur le tableau de bord ; - le responsable corrige avec une régularisation de stock (traçable) ; - en pratique, pendant une coupure, **utiliser un seul poste** pour les sorties de stock réduit fortement ce risque.

7.5 Transport et sécurité

- Un service (sur la machine locale) envoie et reçoit par lots, avec reprise automatique et pas de perte si la connexion tombe en plein envoi.
- Le nœud s'authentifie avec une **clé propre à ce nœud**, limitée à la pharmacie de l'hôpital et **révocable** (si la machine est volée), et non avec la clé `service_role` du cloud.
- Le cloud n'accepte de ce nœud que des lignes de sa pharmacie.
- Un écran « Synchronisation » montre l'état : dernier succès, éléments en attente, éléments en échec avec la raison.

7.6 Ce que la synchronisation ne fera pas

- Elle ne fusionne pas deux modifications simultanées d'une même fiche produit : la plus récente l'emporte.
- Elle ne répare pas un mauvais réglage de l'horloge de la machine : d'où le réglage NTP obligatoire.

8. Plan par phases (estimations indicatives)

Phase	Contenu	Durée
0	**Déjà disponible** : mode hors-ligne des ventes (avec les limites du chapitre 2).	—
1	Nettoyage de la base : mouvements manquants, préfixe de facture par nœud, marque de suppression, journal des changements.	3 à 5 jours
2	**Pilote sans synchronisation** : installer le nœud local sur une machine de test, script d'export filtré de la pharmacie, restauration, démarrage automatique, sauvegardes. Répétition complète de la procédure du chapitre 6.	4 à 6 jours
3	Synchronisation **montante** (local vers cloud) : ventes, mouvements, dépenses, réquisitions. C'est le gain principal : rien ne se perd.	1 à 2 semaines
4	Synchronisation **descendante** (cloud vers local) : produits, lots, paramètres, comptes ; gestion des conflits ; écran de supervision.	1 à 2 semaines
5	Tests de coupure sur copie de la base (coupures pendant un envoi, deux postes en même temps, horloge décalée), puis recette sur site.	1 semaine

Total indicatif : **5 à 7 semaines**. Si l'option A suffit, compter 2 à 3 semaines.

9. Ce dont j'ai besoin pour démarrer

1. **Durée habituelle des coupures** d'internet à l'hôpital (heures ? jours ?).
2. **Fonctions indispensables hors-ligne** : ventes seulement ? réception de stock ? création de produits ? inventaire ? rapports ?
3. **Combien de postes** utilisent la machine locale, et combien de téléphones saisissent des ventes en même temps.
4. **Matériel** réellement disponible (RAM, système, onduleur) et **qui assure la maintenance** sur site.
5. **Accès à une copie de test** de la base (mot de passe de la base Supabase, saisi par vous sur la machine de test, jamais envoyé par message) pour construire et tester sans toucher à la production.
6. Accord sur le **préfixe de facture** propre à l'hôpital.

10. Check-list de recette (tests de coupure)

- [ ] Couper internet : la caisse vend, le stock baisse, un ticket s'imprime.
- [ ] Redémarrer la machine **sans internet** : l'application s'ouvre et on peut se connecter.
- [ ] Recevoir du stock hors-ligne, corriger une quantité, créer un produit.
- [ ] Remettre internet : tout apparaît en ligne sur un téléphone, sans doublon.
- [ ] Vendre sur un téléphone pendant la coupure puis rétablir : les deux ventes sont présentes, aucun numéro de facture en double.
- [ ] Couper internet **en plein envoi** : à la reprise, aucun doublon ni perte.
- [ ] Modifier le même produit des deux côtés : la fiche la plus récente l'emporte, sans plantage.
- [ ] Vendre le dernier article des deux côtés : l'écart est signalé et corrigeable.
- [ ] Débrancher le courant brutalement : redémarrage sans perte de données.

- [ ] Restaurer une sauvegarde sur une autre machine.
- [ ] Désactiver la clé du nœud : le cloud refuse ses envois.

Annexe — Tables de la base (d'après la base en ligne)

32 tables et vues. Tables avec `pharmacy_id` (filtrables par pharmacie) : `audit_logs`, `auth_events`, `customers`, `expenses`, `kit_requisition_items`, `kit_requisitions`, `pharmacy_members`, `pharmacy_opening_hours`, `pharmacy_payment_methods`, `pharmacy_settings`, `pharmacy_subscriptions`, `pricing_rules`, `product_batches`, `product_categories`, `products`, `sale_items`, `sales`, `stock_movements`, `subscription_payments`, `suppliers`.

Sans `updated_at` (donc de type « événement », avec de rares mises à jour d'état) : `sales`, `sale_items`, `stock_movements`, `kit_requisition_items`, `audit_logs`, `auth_events`, `subscription_payments`, `kit_requisitions`.

Fonctions de la base utilisées par l'application et à reproduire à l'identique en local : `create_sale`, `add_product_batch`, `create_product_with_initial_batch`, `correct_batch_quantity`, `remove_batch_from_sale`, `create_kit_requisition`, `cancel_kit_requisition`, `set_sale_payment_method`, `generate_invoice_number`, ainsi que les fonctions de droits (`has_pharmacy_role`, `has_active_pharmacy_role`, `can_sell`, `can_manage_inventory`, `pharmacy_subscription_active`). Ce sont ces fonctions qui garantissent la cohérence du stock : elles sont incluses dans le schéma exporté à l'étape 3.